

Course Outline: Introduction to MCPs

Title: Introduction to MCPs — The Universal Protocol for AI Tools **Skill:** __ — Skill: Proveer de herramientas a un Agente mediante CLIs **Learnpack:** + Introducción a MCPs **File:** s__-w20d59-introduccion-a-mcps **Prerequisite:** CLIs y la IA (subprocess calling, JSON output, exit codes) **Target Audience:** Beginner developers who have built AI-ready CLIs and now want to understand the next evolution of agent tool access **Total Lessons:** 8 **Format:** Conceptual (0% hands-on — implementation is in MCP Server and MCP Client learnpacks)

Course Description

Model Context Protocol (MCP) is an open standard that defines how AI agents discover and call tools — building on the CLI patterns from the previous learnpack but adding type safety, schema validation, and a standardized discovery mechanism. This course explains what MCPs are, the problem they solve, how the client-server architecture works, and what the protocol exchange looks like. No code in this learnpack — the implementation learnpacks (MCP Server in Python, MCP Client in TypeScript) cover that. Students finish with a solid conceptual understanding of WHY MCP exists and HOW it works before writing any code.

Course Philosophy

This course emphasizes:

- Understanding the problem before the solution — why MCP rather than "just use CLIs"
 - Concrete analogies that make abstract protocol concepts stick
 - Preparing students to implement in the next two learnpacks, not implement here
-

Course Statistics Summary

Section	Lessons
00 - Welcome	1
01 - What Is MCP?	2
02 - MCP Architecture	3
03 - Assessment	1
04 - Outro	1
Total	8

00.0 A Universal Protocol for AI Tools 

Learning Objectives:

- Explain the problem that MCP solves in the context of AI tool integration
- Give an analogy that makes the concept of a "universal protocol" concrete

Content Outline:

- The problem without MCP: every AI system integrates with tools differently — OpenAI has function calling, Anthropic has tool use, LangChain has its own format; building a tool means building it 4 different times for 4 different AI systems
- The analogy: MCP is like USB-C — before USB-C, every device had its own charging port; now one cable works everywhere. MCP is the USB-C of AI tool integration
- What MCP promises: build a tool once as an MCP server; any MCP-compatible AI agent (Claude, GPT-4, Gemini) can discover and use it without changes
- Course overview: what MCP is, how it compares to CLIs and APIs, how the architecture works, what the protocol exchange looks like

Transitions: This lesson frames the need. Section 01 goes deep into what MCP is and when to use it versus CLIs or APIs.

Key Principles:

- MCP is a protocol, not a library — it defines a communication format that any language can implement
- The goal is interoperability: one tool, every agent

Examples: Without MCP (before):

- Tool for Claude: implemented in Anthropic tool_use format
- Same tool for OpenAI: reimplemented in function_calling format
- Same tool for LangChain: reimplemented again

With MCP:

- Tool built as MCP server once
- Claude uses it ✓, GPT uses it ✓, any MCP client uses it ✓

01.0 What Is MCP?

Learning Objectives:

- Define MCP (Model Context Protocol) and state who created it and why
- Explain the three core capabilities MCP provides: tools, resources, and prompts
- Preview what lesson 01.1 covers (MCP vs CLI vs API comparison)

Content Outline:

- **What MCP is:** an open protocol (specification) for connecting AI models to external tools, data sources, and capabilities. Created by Anthropic in 2024, now open and community-adopted.

- **Why it was created:** as AI agents became more capable, the need for a standard way to give them tools became critical; without a standard, every integration was custom work
- **The three MCP capabilities:**
 - **Tools:** functions the agent can call — equivalent to what CLIs provide but in a structured, discoverable, typed way
 - **Resources:** data the agent can read — files, database records, API responses exposed as readable context
 - **Prompts:** predefined prompt templates the agent can use — reusable instructions for specific tasks
- **Key distinguisher:** MCP is discoverable — an MCP client asks "what tools do you have?" and the server responds with a schema; with CLIs, the agent must know the command in advance
- Overview of 01.1: when to use MCP vs CLI vs direct API

Transitions: Now you know WHAT MCP is. Lesson 01.1 covers WHEN to use it — the decision framework.

Key Principles:

- MCP is not a replacement for CLIs — it's a higher-level protocol that can wrap CLIs, APIs, databases, and anything else
- The discovery mechanism is what separates MCP from everything before it — the agent learns what's available at runtime

Examples: An MCP server might expose:

- **Tools:** `search_database(query)`, `send_email(to, subject, body)`, `resize_image(path, width, height)`
- **Resources:** `/files/report.csv`, `/db/users`, `/config/settings.json`
- **Prompts:** `summarize_meeting`, `draft_reply_email`

01.1 MCP vs CLI vs Direct API

Learning Objectives:

- Compare MCP, CLIs, and direct API calls across four dimensions: setup complexity, discoverability, type safety, and interoperability
- Choose the appropriate integration approach for a given scenario
- Explain why MCP is not always the right choice (overkill for simple integrations)

Content Outline:

- **CLI approach (from previous learnpack):**
 - Setup: easy — any program with stdin/stdout
 - Discoverability: none — agent must know the command
 - Type safety: none — output is untyped text (even JSON)
 - Interoperability: universal — any agent can run a shell command
 - When to use: existing CLI tools, quick integrations, simple one-way calls
- **Direct API approach:**

- Setup: requires HTTP client, auth, endpoint knowledge
- Discoverability: sometimes via OpenAPI spec, often manual
- Type safety: sometimes via schema, often not
- Interoperability: agent-specific (different syntax per AI system)
- When to use: existing APIs, when MCP is overkill
- **MCP approach:**
 - Setup: requires MCP server implementation (the next two learnpacks)
 - Discoverability: automatic — clients ask servers what they expose
 - Type safety: built-in — tool schemas define types, validation is automatic
 - Interoperability: universal among MCP clients — build once, use everywhere
 - When to use: tools you want to reuse across multiple AI systems, production-grade integrations, complex tools with many capabilities
- **Decision framework:**
 - One AI system, simple task → CLI or API
 - Multiple AI systems, or complex tool → MCP

Transitions: You know when to use MCP. Section 02 explains how it works architecturally.

Key Principles:

- MCP adds overhead — it's not always the right choice for simple, single-use integrations
- When you're building for the ecosystem (multiple AI systems, or open-source tools), MCP is the right choice

Anti-patterns:

- **Using MCP for a tool only one agent will ever call:** unnecessary complexity
- **Using CLIs when you need type safety:** errors from wrong types won't be caught until runtime

Examples:

Scenario	Best approach
Run a bash script from Claude	CLI
Call your company's internal API from one chatbot	Direct API
Build a weather tool for any MCP-compatible agent	MCP
Expose a database to multiple AI assistants	MCP
Quick file processing script for a prototype	CLI

02.0 MCP Architecture

Learning Objectives:

- Describe the MCP client-server model and how messages flow
- Explain what a "host" is in MCP terminology
- Preview the roles covered in lessons 02.1 (clients and servers) and 02.2 (the protocol standard)

Content Outline:

- **The client-server model:** MCP is built on a client-server architecture where:
 - **MCP Server:** exposes tools, resources, and prompts — it's the capability provider
 - **MCP Client:** connects to servers and makes them available to AI models — it's the agent-side component
 - **Host:** the application that contains the MCP client — e.g., Claude Desktop, a custom chatbot app
- **The relationship:**
 - Host (your app) → contains MCP Client → connects to → MCP Server (your tool)
 - AI model → asks MCP Client for tools → MCP Client queries server → server responds with tool list
- **Message flow overview:**
 1. Client connects to server (handshake)
 2. Client requests capabilities ("what tools do you have?")
 3. Server responds with tool list + schemas
 4. AI model selects a tool and provides arguments
 5. Client calls the tool on the server
 6. Server executes and returns a result
 7. Client forwards result to the AI model
- Overview of 02.1 (clients and servers in depth) and 02.2 (the protocol exchange)

Transitions: This overview shows the architecture. Lesson 02.1 goes deep into the client and server roles individually.

Key Principles:

- The host/client/server separation allows one server to be used by multiple hosts simultaneously
- The client is the intermediary — the AI model never talks directly to the MCP server

Examples: Claude Desktop as host:

- Claude Desktop (host) runs an MCP client
- MCP client connects to: database server, filesystem server, web search server
- Claude can now use all three simultaneously
- User asks: "Search for articles about Python, save the top result to my downloads"
- Claude calls web_search tool → saves result using filesystem tool

02.1 MCP Clients and Servers

Learning Objectives:

- Describe the specific responsibilities of an MCP server vs an MCP client
- Give examples of real-world MCP clients (Claude Desktop, Cursor, Continue.dev) and servers
- Explain how the connection between client and server is established (transport layer)

Content Outline:

- **MCP Servers — what they do:**
 - Define and expose tools (with full JSON schemas for inputs and outputs)
 - Expose resources (readable data)
 - Expose prompt templates
 - Handle tool calls when invoked
 - Run in a separate process (or even a remote server)
 - Can be built in any language — the protocol defines communication, not implementation
- **MCP Clients — what they do:**
 - Connect to one or more MCP servers
 - Request the list of available tools, resources, prompts
 - Forward tool calls from the AI model to the appropriate server
 - Return results to the AI model
 - Manage the connection lifecycle (connect, disconnect, reconnect)
- **Real-world MCP clients:**
 - Claude Desktop: Anthropic's desktop app, natively supports MCP
 - Cursor: AI code editor with MCP support
 - Continue.dev: open-source AI coding assistant with MCP support
 - Custom implementations: you can build your own MCP client (the next learnpack)
- **Transport layer:** how client and server communicate
 - **stdio transport:** client spawns server as a subprocess, communicates via stdin/stdout (most common for local tools)
 - **SSE (Server-Sent Events) transport:** HTTP-based, for remote servers
 - The transport layer is abstracted — the protocol messages are the same regardless of transport

Transitions: You know the roles. Lesson 02.2 explains what the actual protocol messages look like.

Key Principles:

- A server knows nothing about the AI model or the host — it only knows about the client that called it
- One MCP server can serve multiple clients simultaneously

Examples: stdio transport in practice:

- Client spawns: `python my_tool_server.py`
- Server starts and waits for messages on stdin
- Client sends: `{"jsonrpc": "2.0", "method": "tools/list", "id": 1}`
- Server responds: `{"jsonrpc": "2.0", "result": {"tools": [...]}, "id": 1}`
- All communication happens over stdin/stdout pipes

02.2 The MCP Standard

Learning Objectives:

- Describe the JSON-RPC 2.0 foundation of MCP messages
- Explain the MCP initialization handshake and capability negotiation

- Describe what a tool schema looks like and how the client uses it to validate calls

Content Outline:

- **The protocol foundation: JSON-RPC 2.0**
 - MCP uses JSON-RPC 2.0 as its message format — a standard for remote procedure calls over JSON
 - Every message has: `jsonrpc: "2.0"`, a `method`, optional `params`, and an `id` for matching responses
 - Request: `{"jsonrpc": "2.0", "method": "tools/call", "params": {...}, "id": 5}`
 - Response: `{"jsonrpc": "2.0", "result": {...}, "id": 5}`
 - Error: `{"jsonrpc": "2.0", "error": {"code": -32603, "message": "..."}, "id": 5}`
- **The initialization handshake:**
 1. Client sends: `initialize` with its capabilities
 2. Server responds: `initialize` result with its capabilities
 3. Client sends: `initialized` notification (handshake complete)
- **Tool discovery: `tools/list`**
 - Client sends: `{"method": "tools/list"}`
 - Server responds with: array of tool objects, each containing: `name`, `description`, JSON Schema for `inputSchema`
- **Tool invocation: `tools/call`**
 - Client sends: `{"method": "tools/call", "params": {"name": "search", "arguments": {"query": "python"}}}`
 - Server validates arguments against schema, executes, returns result
- **The schema advantage:** the JSON Schema for each tool lets the client validate arguments BEFORE calling — prevents errors from reaching the server

Transitions: With the protocol understood, the assessment checks comprehension of the full MCP picture.

Key Principles:

- JSON-RPC 2.0 is a well-established standard — MCP inherits its request/response matching and error handling
- Tool schemas enable automatic validation — a client that respects the schema can't send invalid arguments

Examples: A complete tool schema:

```
{
  "name": "search_files",
  "description": "Search for files by name or content",
  "inputSchema": {
    "type": "object",
    "properties": {
      "query": {"type": "string", "description": "Search term"},

```

```
    "path": {"type": "string", "description": "Directory to search in"}
  },
  "required": ["query"]
}
```

The client knows: **query** is required (string), **path** is optional (string). Any call without **query** is rejected before reaching the server.

03.0 MCP Assessment 🧠

Learning Objectives:

- Demonstrate understanding of MCP concepts: purpose, architecture, protocol
- Correctly identify when MCP is and isn't the right choice
- Apply client/server/host terminology correctly

Question Format: Multiple choice

Questions:

1. What is the primary problem that MCP solves?
 - a) It makes CLI tools faster
 - b) It provides a universal protocol for AI agents to discover and use tools, eliminating the need to integrate tools differently for each AI system
 - c) It replaces all existing APIs with a new standard
 - d) It allows AI models to run code directly without any external tools
2. Which analogy best describes what MCP does for AI tool integration?
 - a) MCP is like email — it allows sending messages between systems
 - b) MCP is like USB-C — one standard that works with any compatible device, eliminating proprietary connectors
 - c) MCP is like WiFi — it provides wireless connectivity
 - d) MCP is like a database — it stores and retrieves data
3. What are the three capabilities an MCP server can expose?
 - a) Models, datasets, and training pipelines
 - b) Tools, resources, and prompts
 - c) Functions, APIs, and webhooks
 - d) Commands, outputs, and errors
4. In MCP terminology, what is the "host"?
 - a) The remote server that runs tools
 - b) The AI model that makes decisions
 - c) The application that contains the MCP client (e.g., Claude Desktop)

- d) The database that stores tool schemas
5. What makes MCP tools different from CLI tools in terms of discoverability?
- a) MCP tools have no discovery — agents must know them in advance
 - b) MCP tools require reading documentation to discover
 - c) MCP clients can ask servers what tools are available at runtime — agents discover tools dynamically
 - d) CLI tools are more discoverable than MCP tools
6. An MCP server uses stdio transport. What does this mean?
- a) The server communicates over HTTP with SSL
 - b) The client spawns the server as a subprocess and communicates via stdin/stdout pipes
 - c) The server uses a database for storing messages
 - d) Communication happens via WebSockets
7. What protocol format does MCP use for its messages?
- a) REST/HTTP with XML
 - b) GraphQL over HTTP
 - c) JSON-RPC 2.0
 - d) Protocol Buffers (protobuf)

Evaluation Criteria:

- 7/7: Excellent — strong grasp of MCP concepts and architecture
- 5–6/7: Good — minor gaps in transport or protocol details
- 3–4/7: Needs review — revisit Sections 01 and 02
- Below 3/7: Restart recommended

Score Interpretation: Questions map to: MCP purpose (Q1), USB-C analogy (Q2), three capabilities (Q3), host definition (Q4), discoverability (Q5), stdio transport (Q6), JSON-RPC (Q7).

04.0 The Standard That Connects Everything

Content Outline:

- Course recap: what students accomplished
 - Understood the problem MCP solves: tool fragmentation across AI systems
 - Learned MCP's three capabilities: tools, resources, prompts
 - Compared MCP vs CLI vs direct API and identified when each is appropriate
 - Traced the client-server-host architecture and message flow
 - Understood the JSON-RPC 2.0 protocol foundation, tool schemas, and discovery
- Connection to bigger picture: MCP is the foundation for the next two learnpacks — MCP Server (implement one in Python) and MCP Client (implement one in TypeScript). Everything covered here prepares you to write real code.
- Next steps: MCP Server learnpack — build a Python server that exposes tools via MCP; then MCP Client learnpack — build a TypeScript client that connects to that server

- Resources for further exploration:
 - MCP specification: spec.modelcontextprotocol.io
 - MCP GitHub: github.com/modelcontextprotocol
 - Anthropic's MCP documentation: docs.anthropic.com/claude/docs/mcp
 - List of community MCP servers: github.com/modelcontextprotocol/servers
- Encouragement: you've just learned a protocol that is being adopted across the AI industry — understanding MCP at this level puts you ahead of the vast majority of developers

Note: This is conclusion only — no theory, exercises, or assessment.
